

STUDENT PERFORMANCE ANALYZER

PROJECT REPORT

Submitted By

Name: Manas Pandey

Registration ID: 26BET10001

Branch: CSE – Educational Technology

Project: Student Performance Analyzer

Programming Language: Python

1. Introduction

Student Performance Analyzer is a command-line based Python application that manages and analyzes basic student academic information.

In the context of the education domain, maintaining student marks/attendance and analyzing their performance are integral. The project aims at developing a basic application that handles these tasks using fundamental Python programming concepts.

The application empowers users to register students, mark subject-wise scores, record attendance, generate reports, and analyze performance at the class level.

2. Problem Statement

Student academic records handling can become monotonous and tedious.

A simple system is required to:

Store student information

Record subject-wise scores

Calculate averages

Assign grades

Compute attendance percentage

Identify best and weakest subjects

Analyze class performance

These requirements have been fulfilled by developing a menu-driven Student Performance Analyzer.

3. Objectives

The objectives of the project are:

1. Develop a student performance management system using Python.
2. Store student information using dictionaries.
3. Automatically calculate student averages and grades.
4. Compute attendance percentage.
5. Generate individual performance reports.
6. Identify best and weakest students.
7. Calculate overall class average.
8. Practice functions, loops, conditions, dictionaries, and exception handling.
9. Practice developing an application using fundamental programming concepts.

4. Scope of the Project

The project currently focuses on a basic level of academic performance management.

The system offers the following:

Student registration

Marks management

Attendance management

Student performance report

Student listing

Class performance analysis

The project aims at providing a baseline for extending and building a full-fledged academic management system.

5. Technologies Used

Technology	Purpose
-----	-----
Python	Application development
Visual Studio Code	Development environment
Command Line Interface	User interaction
Python Dictionaries	Data storage
GitHub	Project repository and submission

No external Python libraries are required.

6. System Requirements

Hardware Requirements

Computer or laptop

Recommended: 4 GB RAM

Keyboard and display

Software Requirements

Windows, Linux, or macOS

Python 3.x

Visual Studio Code or any compatible Python editor

Command-line terminal

7. Functional Requirements

The application has the following functions.

7.1 Add Student

Users can input:

Roll number

Name

Branch

Semester

Roll number acts as a unique student identifier.

7.2 Add Marks

User can enter:

Number of subjects

Subject names

Marks for each subject

The application validates that marks are between 0 and 100.

7.3 Add Attendance

User can enter:

Total number of classes

Number of classes attended

The application automatically calculates attendance.

7.4 Student Report

This feature generates a student report that contains:

Student information

Subject marks

Total marks

Average percentage

Grade

Attendance

Attendance status

Best subject

Weakest subject

Overall performance

7.5 Display Students

This function displays all students along with their roll number, name, and branch.

7.6 Performance Analysis

This function analyzes the performance and determines:

Highest-performing student

Lowest-performing student

Class average

8. System Design

The application follows a simple function-based design.

The major functions are as follows:

```
```text
```

```
main()
```

```
|
|— add_student()
|— add_marks()
|— add_attendance()
|— student_report()
|— display_students()
|— performance_analysis()
...

```

Additional helper functions are as follows:

```
```text
grade()
attendance()
...

```

The `main()` function provides a menu and manages the application flow.

```
---
```

9. Data Structure

The project uses a nested dictionary.

The main dictionary is:

```
```python
students = {}
...

```

Each roll number acts as a key.

Each student contains information such as:

```
```python
{
    "name": "...",
    "branch": "...",
    "semester": "...",
    "subjects": {},

```

```

"total": 0,
"attended": 0
}
...

```

The nested `subjects` dictionary stores subject names and their marks.

This structure enables storing and retrieving student information and academic data efficiently within the program.

```
---
```

10. Grade Calculation

The program assigns grades based on the student's average percentage.

The grading system is:

Average	Grade
90 and above	A+
80–89	A
70–79	B+
60–69	B
50–59	C
40–49	D
Below 40	F

The `grade()` function uses conditional statements to determine the grade.

```
---
```

11. Attendance Calculation

Attendance is calculated using:

```
```text
```

Attendance =

$(\text{Attended Classes} / \text{Total Classes}) \times 100$

```
...
```

Example: If a student attends 36 out of 40 classes,

```
```text
```

Attendance = $(36 / 40) \times 100$

= 90%

```
```
```

The program considers attendance of 75% and above as Good and attendance below 75% as Warning.

```

```

## # 12. Performance Calculation

The average percentage is calculated from the subject marks.

Example: If a student has marks:

```
```text
```

Mathematics = 80

Programming = 90

Physics = 70

```
```
```

Then:

```
```text
```

Total = $80 + 90 + 70$

= 240

Average = $240 / 3$

= 80%

```
```
```

The grade is then calculated using the grade function.

```

```

## # 13. Performance Classification

The program classifies performance using the following categories:

| Percentage | Classification |
|------------|----------------|
|------------|----------------|

|        |       |
|--------|-------|
| -----: | ----- |
|--------|-------|

|               |           |
|---------------|-----------|
| 75% and above | Excellent |
|---------------|-----------|

| 60%–74% | Good |  
| 50%–59% | Average |  
| 40%–49% | Needs Improvement |  
| Below 40% | Poor |

---

## # 14. Input Validation

The project has several input validations.

The program checks:

### ### Duplicate Student

If the entered roll number already exists, the program displays:

```
```text
```

Student already exists

```
```
```

### ### Invalid Marks

Rejects marks outside the range 0–100.

### ### Invalid Numerical Input

The program uses `try-except` to handle invalid numerical input.

### ### Invalid Attendance

Rejection conditions:

Negative total classes

Negative attended classes

Attended classes greater than total classes

---

## # 15. Algorithm

### ## Algorithm for Adding a Student

1. Start.
2. Ask the user for the roll number.
3. Check whether the roll number already exists.
4. If it exists, display an error.

5. Otherwise, ask for student details.
6. Create the student's dictionary.
7. Store it in the `students` dictionary.
8. Display confirmation.
9. Stop.

---

### ## Algorithm for Adding Marks

1. Start.
2. Ask for the student's roll number.
3. Check whether the student exists.
4. Ask for the number of subjects.
5. Enter each subject name.
6. Enter marks.
7. Validate marks between 0 and 100.
8. Store the marks.
9. Save the subject dictionary.
10. Stop.

---

### ## Algorithm for Generating a Student Report

1. Start.
2. Ask for roll number.
3. Search for the student.
4. Display student information.
5. Check whether marks are available.
6. Calculate total marks.
7. Calculate average.
8. Determine grade.
9. Calculate attendance.
10. Determine attendance status.

11. Find best and weakest subjects.
12. Determine performance category.
13. Display the report.
14. Stop.

---

## # 16. Program Flow

```text

START

|

v

Display Main Menu

|

v

Enter Choice

|

+---- 1 ----> Add Student

|

+---- 2 ----> Add Marks

|

+---- 3 ----> Add Attendance

|

+---- 4 ----> Student Report

|

+---- 5 ----> Display Students

|

+---- 6 ----> Performance Analysis

|

+---- 7 ----> Exit

|

v

Return to Menu

...

17. Testing

The application can be tested using different input cases.

| Test Case | Input/Condition | Expected Result |
|----------------------|----------------------|----------------------------|
| ----- | ----- | ----- |
| Add student | New roll number | Student added |
| Duplicate student | Existing roll number | Student already exists |
| Valid marks | 85 | Marks accepted |
| Invalid marks | 110 | Marks rejected |
| Valid attendance | 36/40 | 90% |
| Invalid attendance | 45/40 | Invalid attendance |
| Missing student | Unknown roll number | Student not found |
| No students | Empty database | No students registered |
| Performance analysis | Students with marks | Class statistics displayed |

18. Sample Output

```text

### STUDENT PERFORMANCE ANALYZER

-----

1. Add Student
2. Add Marks
3. Add Attendance
4. Student Report
5. Display Students
6. Performance Analysis
7. Exit

Enter choice: 1

Roll Number: 101

Name: Rahul

Branch: CSE

Semester: 1

Student added!

...

After entering marks and attendance, a report can be generated:

```text

STUDENT REPORT

=====

Name: Rahul

Roll No.: 101

Branch: CSE

Semester: 1

Mathematics: 85.00

Programming: 92.00

Physics: 78.00

Total: 255.00

Average: 85.00%

Grade: A

Attendance: 90.00%

Attendance Status: Good

Best Subject: Programming

Weakest Subject: Physics

Performance: Excellent

...

19. Advantages

The project offers several advantages:

Simple and easy-to-use interface.

Automates basic calculations.

Reduces manual calculation errors.

Demonstrates practical Python programming.

Uses functions for modularity.

Provides input validation.

Provides both individual and class-level analysis.

Can be expanded into a larger academic management system.

20. Limitations

The current implementation has some limitations:

1. Data is stored only in memory.
2. Data is lost when the program is closed.
3. No database.
4. No graphical user interface.
5. No user authentication.
6. Student records cannot currently be edited or deleted.
7. Reports cannot currently be exported to files.

21. Future Scope

The project can be improved in future versions by implementing:

JSON or CSV file storage

SQLite database

GUI using Tkinter

Student search functionality

Edit and delete operations

PDF report generation

Graphical performance charts

Login and authentication

Subject-wise analytics

Semester-wise analytics

Persistent student records

Automated report generation

22. Learning Outcomes

The project helped demonstrate practical knowledge of:

Python syntax

Variables and data types

Functions

Dictionaries

Nested dictionaries

Loops

Conditional statements

Exception handling

Input validation

Mathematical calculations

Lambda functions

Built-in functions

Menu-driven programming

Basic data analysis

Modular program design

23. Conclusion

The Student Performance Analyzer successfully demonstrates the implementation of a practical command-line application using fundamental Python programming concepts.

The application provides functionality for student registration, marks management, attendance calculation, individual reporting, and class performance analysis.

Although the current version uses temporary in-memory storage, it provides a strong foundation for developing a more advanced academic management system. Future versions can incorporate databases, graphical interfaces, persistent storage, and advanced analytics.

24. References

1. Python official documentation
2. Python programming concepts related to dictionaries, functions, loops, and exception handling
3. Visual Studio Code documentation

25. Project Information

Project Title: Student Performance Analyzer

Student Name: Manas Pandey

Registration ID: 26BET10001

Branch: CSE – Educational Technology

Programming Language: Python

Project Type: Command-Line Application